WPT Day 12

Sun 01 Nov 26 · The `isLoading` state pattern (and the `users.length` loading bug), class component lifecycle functions, `useEffect` as the lifecycle of function components, the first-render rule, the `key` prop, memoization (`memo` , `useMemo` , `useCallback`), the "three-tool" performance pattern, the full-stack project build order (entities → schema → UI operations → APIs → frontend), and learning advice

---

Table of Contents

1. isLoading State and the Conditional Rendering Bug
2. Class Component Lifecycle Functions
3. useEffect: Lifecycle in Function Components
4. useEffect First-Render Behaviour
5. The key Prop in List Rendering
6. Memoization: Concept and Purpose
7. memo: Preventing Unnecessary Child Re-Renders
8. useMemo: Memoizing Expensive Calculations
9. useCallback: Memoizing Functions Passed as Props
10. The Three-Tool Pattern and Dashboard Performance
11. Full-Stack Project Build Order
12. Child-to-Parent Data, Next Steps and Advice
13. Combined Worked Example
14. Quick Revision Checklist
15. Practice Exercises
16. Practice Answers

---

isLoading State and the Conditional Rendering Bug

Problem: using `users.length` as a loading indicator

Yesterday's `UsersList` decided whether to show a "Loading" message with the condition `users.length === 0` . The assumption was: *the array is empty only while the API call is in progress, and it gets filled once data arrives.*

That assumption breaks the moment the **server legitimately returns an empty array**:

- The UI shows "Loading..." **indefinitely**, even though the API has already responded.
- The user gets a **false signal** — the app looks stuck, but the fetch is complete.
- **Root cause:** `users.length === 0` is true **both** while fetching **and** when the server simply has no data. One condition is trying to answer two different questions.

Mechanism: a dedicated `isLoading` state

Introduce a **separate boolean state** that tracks only one thing: *is the API call still in progress?*

- `isLoading` is **independent** of whether data exists.
- It is `true` from the moment the component mounts until the API responds.
- It is set to `false` inside `setState` once the response arrives — **regardless of whether the data is empty or not**.

Step-by-step execution flow

1. Component mounts → `isLoading: true`, `users: []`.
2. `componentDidMount` fires → the API call begins.
3. While the API is pending → show "Loading..." (because `isLoading === true`).
4. API responds (even with an empty array) → `setState({ users: data, isLoading: false })`.
5. Re-render is triggered:
   - If `isLoading === false` **and** `users.length === 0` → show **"No users found"**.
   - If `isLoading === false` **and** `users.length > 0` → render the table rows.

```
mount ──► isLoading = true,  users = []        ──► render: "Loading..."
            │
            │  componentDidMount → fetch(...)   (API pending)
            ▼
response ──► setState({ users: data, isLoading: false })
            │
            ├── users.length === 0 ──► render: "No users found"
            └── users.length  > 0  ──► render: table rows (map)
```

Lecture's code example

The lecture wraps the fetch in a `setTimeout` of 3 seconds so the "Loading..." screen is visible long enough to see.

```jsx
import { Component } from "react";

class UsersList extends Component {
  constructor(props) {
    super(props);
    this.state = {
      users: [],
      isLoading: true, // true from the start because fetch begins on mount
    };
  }

  componentDidMount() {
    setTimeout(() => {
      fetch("https://jsonplaceholder.typicode.com/users")
        .then((response) => response.json())
        .then((data) => {
          this.setState({ users: data, isLoading: false });
        });
    }, 3000);
```

```
  }

  render() {
    const { users, isLoading } = this.state;

    if (isLoading) {
      return <h4>Loading ... </h4>;
    }

    return (
      <div>
        <h2>Users List</h2>
        <table border="1" cellPadding="5" cellSpacing="0">
          <thead>
            <tr>
              <th>ID</th>
              <th>Name</th>
              <th>Username</th>
              <th>Phone</th>
              <th>Email</th>
            </tr>
          </thead>
          <tbody>
            {users.length === 0 ? (
              <tr>
                <td colSpan="5">No users found</td>
              </tr>
            ) : (
              users.map((user) => (
                <tr key={user.id}>
                  <td>{user.id}</td>
                  <td>{user.name}</td>
                  <td>{user.username}</td>
                  <td>{user.phone}</td>
                  <td>{user.email}</td>
                </tr>
              ))
            )}
          </tbody>
        </table>
      </div>
    );
  }
}

export default UsersList;
```

Key insight

- `isLoading` controls whether to show the **loading message**.
- `users.length` controls whether to show **data or "No users found"**.

- These are **two separate concerns** and must be tracked with **two separate states**.

> **Pitfall:** Never use `array.length === 0` as the "still loading" signal. An empty array is a valid server answer; "loading" and "empty" are different situations and need different state.

> **Exam:** Why add an `isLoading` state when `users.length` already exists? Because `users.length === 0` is true both during fetching and when the server returns no data — only a dedicated boolean can tell "waiting" apart from "finished with nothing".

Class Component Lifecycle Functions

Problem: how do we run code at specific moments of a component's life?

A component is not static. It **appears** on screen, it **changes** when state or props update, and it **disappears**. We often need to do work at exactly those moments — fetch data when it appears, sync with a backend when it changes, clean up when it goes away. React needs a way to say "call my code at moment X".

Mechanism

Lifecycle functions are predefined methods inherited from React's `Component` class. React **automatically calls** them at specific points in a component's life: when it appears on screen (**mounts**), when it updates, and when it disappears (**unmounts**). They are a **native feature of class components only**.

The full lifecycle sequence

```
constructor()
     ↓
render()                    ← first render
     ↓
componentDidMount()
     ↓
[ user interaction / state update ]
     ↓
render()                    ← re-render
     ↓
componentDidUpdate()
     ↓
[ component removed from DOM ]
     ↓
componentWillUnmount()

[ at any point, if an error occurs ]
     ↓
componentDidCatch()
```

Each lifecycle method explained

Method	When it is called	Typical use
`constructor(props)`	First, **once**, when the component is being created	Initialise state
`render()`	After the constructor (first time) **and** after every state/prop update	Return JSX — keep it **pure**, no side effects
`componentDidMount()`	**Once**, just after the first render	Fetch data, set up subscriptions, start timers
`componentDidUpdate(prevProps, prevState)`	After **every re-render** caused by a state/prop update; **not** after the first render	Sync frontend state to backend (e.g. update score via API)
`componentWillUnmount()`	Just **before** the component is destroyed and removed from the DOM	Cancel pending API calls, clear timers, close connections
`componentDidCatch(error, info)`	When an error occurs anywhere inside this component's tree	Show a fallback UI via an ErrorBoundary

```
import { Component } from "react";

class LifecycleDemo extends Component {
  constructor(props) {
    super(props);
    // Called first, only once, when component is being created.
    // Initialize state here.
    this.state = { score: 0 };
  }

  render() {
    // Called after constructor (first time) and after every state update.
    // Must return JSX. Keep this pure - no side effects here.
    return <div>Score: {this.state.score}</div>;
  }

  componentDidMount() {
    // Called once, just after the first render.
    // Best place to fetch data, set up subscriptions, start timers.
    console.log("Component mounted - fetch data here");
  }

  componentDidUpdate(prevProps, prevState) {
    // Called after every re-render caused by a state or prop update.
    // NOT called after the first render.
    // Use case: sync frontend state to backend (e.g. update score via API)
```

```
    if (prevState.score !== this.state.score) {
      console.log("Score changed - send API call to update backend");
    }
  }

  componentWillUnmount() {
    // Called just BEFORE the component is destroyed and removed from the DOM.
    // Use case: cancel pending API calls, clear timers, close connections.
    console.log("Component about to be destroyed - clean up here");
  }

  componentDidCatch(error, info) {
    // Called when an error occurs anywhere inside this component's tree.
    // Use with ErrorBoundary to show a fallback UI instead of a blank screen.
    console.log("Error caught:", error);
  }
}

export default LifecycleDemo;
```

Background: `componentDidUpdate` receives the **previous** props and state. Comparing them (`prevState.score !== this.state.score`) is what stops an API call from firing on *every* update — only when the value you care about actually changed.

The `componentWillUnmount` analogy

The instructor used the `finalize` method from Java/JVM:

> "J for JVM, J for **Jallad** (executioner). When the executioner is about to hang someone, he first asks for their **last wish**. The `finalize` method fulfills that last wish before the object is destroyed."

`componentWillUnmount` is React's **"last wish"** mechanism.

Real-world scenario (lecture):

- The user is on the **Home** page, which starts a slow API call (10–20 seconds, poor network).
- Before the API responds, the user navigates to the **About Us** page.
- The Home component is **destroyed**, but the browser keeps executing the pending API call — wasting resources and potentially causing errors when it tries to update state on an unmounted component.
- **Solution:** inside `componentWillUnmount`, **cancel any pending API requests** and close any open connections.

`componentDidCatch` and ErrorBoundary

- **Without** error handling: any component crash causes a **blank white screen**.

- **With** `componentDidCatch` + ErrorBoundary: only the **crashed component** shows a fallback UI; the rest of the app (navbar, footer, other components) **keeps working normally**.
- Real-world example (lecture): **Amazon** shows a random animal image with "Oops!" when a section fails, rather than crashing the entire page.
- `componentDidCatch` will be explored in depth **in a later session** alongside the ErrorBoundary concept.

> **Exam:** Lifecycle order: `constructor` → `render` → `componentDidMount` → (state update) → `render` → `componentDidUpdate` → `componentWillUnmount`; `componentDidCatch` fires on an error at any point. `componentDidUpdate` is **not** called after the first render.

---

useEffect: Lifecycle in Function Components

Problem: function components cannot inherit lifecycle methods

Lifecycle functions (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`) are native to **class components** — they exist because a class **inherits** from `Component`. A **function component does not inherit from anything**, so these methods are simply not available.

- Writing `componentDidMount()` inside a function component **does nothing**.
- React provides the `useEffect` **hook** as the **single unified mechanism** to implement all lifecycle phases in function components.

Mechanism: one hook, all phases

A single `useEffect` can behave like `componentDidMount`, `componentDidUpdate` or `componentWillUnmount` depending on how it is configured. The configuration is controlled by the **dependency array** (second argument).

```
useEffect(callback, dependencyArray);
```

As `componentDidMount` — empty dependency array

- Pass an empty array `[]` as the second argument.
- The callback runs **once**, just after the first render.
- It will **not** run again on subsequent state updates.

```
useEffect(() => {
  console.log("componentDidMount equivalent – runs once after first render");
  // Fetch data, set up subscriptions, etc.
}, []); // ⟵ empty array is the key
```

As `componentDidUpdate` — with dependencies

- Pass one or more state variables in the dependency array.
- The callback runs after the first render **and again** whenever any listed dependency updates.

```
useEffect(() => {
  console.log("componentDidUpdate equivalent - runs when x or y changes");
}, [x, y]); // ⟵ runs when x or y state updates
```

As `componentWillUnmount` — return a cleanup function

- **Return a function** from inside the `useEffect` callback.
- That returned function is called **just before the component is destroyed**.
- The rest of the `useEffect` code runs normally based on the dependency array; **only the returned function** is tied to unmounting.

```
useEffect(() => {
  console.log("This runs based on dependency array as usual");

  return () => {
    // Only THIS returned function runs when the component is destroyed
    console.log("componentWillUnmount equivalent - cleanup here");
    // Cancel API calls, clear timers, close WebSocket connections
  };
}, []); // Can have dependencies or be empty - doesn't matter for the cleanup
```

- **Important:** the entire `useEffect` does **not** run on unmount — **only the returned function** does.
- You do **not** need a separate `useEffect` for cleanup; add the return function to any existing `useEffect`.

Comparison: lifecycle method vs `useEffect`

Class lifecycle method	`useEffect` equivalent	Dependency array
`componentDidMount`	`useEffect(() => { ... }, [])`	Empty `[]`
`componentDidUpdate`	`useEffect(() => { ... }, [dep1, dep2])`	Listed state variables
`componentWillUnmount`	`return () => { ... }` inside any `useEffect`	Any (or empty)
`componentDidCatch`	ErrorBoundary + `componentDidCatch`	Class component only

The async `useEffect` pitfall (common error)

A very common mistake is to mark the `useEffect` callback itself as `async`:

```
// WRONG - do NOT do this
useEffect(async () => {
  const response = await fetch("...");
}, []);
// Warning: "Effect callbacks are synchronous to prevent race conditions.
// Put the async function inside."
```

Why is this wrong? An `async` function **always returns a Promise**. But React's rule is that a `useEffect` callback can return **only a function (for cleanup) or nothing**. Returning a Promise violates that rule.

The fix: define a separate `async` function **inside** the effect and call it.

```
// CORRECT
useEffect(() => {
  const fetchUsersData = async () => {
    try {
      const response = await fetch("https://jsonplaceholder.typicode.com/users");
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      console.log(error);
    }
  };

  fetchUsersData(); // call the async function inside the effect
}, []);
```

The invalid return value error

```
// WRONG - returning a number
useEffect(() => {
  return 7;
  // Error: "useEffect must not return anything besides a function
  // which is used for cleanup, but you have returned 7."
}, []);
```

React throws a clear error if you return anything other than a function. **Only a function (or nothing) is a valid return** from a `useEffect` callback.

Full function component example with data fetching

```
import { useState, useEffect } from "react";

function UsersData() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
```

```
    const fetchUsersData = async () => {
      try {
        const response = await fetch("https://jsonplaceholder.typicode.com/users");
        const data = await response.json();
        setUsers(data);
      } catch (error) {
        console.log(error);
      }
    };

    fetchUsersData();
  }, []); // empty array = componentDidMount equivalent

  return (
    <div>
      <h2>List of Users</h2>
      <table border="1" cellPadding="5" cellSpacing="0">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Username</th>
            <th>Phone</th>
            <th>Email</th>
          </tr>
        </thead>
        <tbody>
          {users.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.username}</td>
              <td>{user.phone}</td>
              <td>{user.email}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default UsersData;
```

Background: The same `isLoading` idea from the first section applies here too — add `const [isLoading, setIsLoading] = useState(true);` and call `setIsLoading(false)` after `setUsers(data)` (the full example at the end of these notes does exactly that).

> **Pitfall:** `useEffect(async () => { ... }, [])` is wrong because an `async` callback returns a Promise, and a `useEffect` callback may only return a cleanup **function** or nothing. Put the `async` function **inside** the effect and call it.

---

useEffect First-Render Behaviour

The surprising rule

On the **very first render** (mounting phase), **all `useEffect` hooks in a component run** — regardless of whether their dependency array is empty or contains values.

```
import { useState, useEffect } from "react";

function MyComponent() {
  const [x, setX] = useState(100);
  const [y, setY] = useState(500);

  // Effect 1: empty array (componentDidMount equivalent)
  useEffect(() => {
    console.log("didMount - runs after first render only");
  }, []);

  // Effect 2: with dependencies (componentDidUpdate equivalent)
  useEffect(() => {
    console.log("didUpdate - x or y has updated. x:", x, "y:", y);
  }, [x, y]);

  return (
    <div>
      <p>x is {x} and y is {y}</p>
      <button onClick={() => setX(x + 5)}>Change X</button>
      <button onClick={() => setY(y + 10)}>Change Y</button>
    </div>
  );
}

export default MyComponent;
```

What happens step by step

1. **First render:** both Effect 1 **and** Effect 2 run — even though `x` and `y` have not "updated" yet, Effect 2 still runs on mount.
2. User clicks **Change X:** only Effect 2 runs (`x` is in its dependency array).
3. User clicks **Change Y:** only Effect 2 runs (`y` is in its dependency array).
4. Effect 1 **never runs again** after the first render.

Event	Effect 1 `[]`	Effect 2 `[x, y]`
First render (mount)	runs	runs
Click "Change X"	—	runs
Click "Change Y"	—	runs
Parent re-renders, `x` and `y` unchanged	—	—

> **Exam:** A `useEffect` with dependencies is **not** a pure "didUpdate": it also runs once on mount. That is the difference from the class `componentDidUpdate`, which is skipped on the first render.

Student Q&A: why keep both effects instead of using a flag?

A student asked: *"Can't we just use a flag (like a render counter) to check if it's the first render, and use only one `useEffect`?"*

Instructor's answer:

> "You could, but then you'd need to track the render count, check if it's the first or second render, and that creates messy code. It's much cleaner to have one `useEffect` with an empty array for mount-only logic, and a separate `useEffect` with dependencies for update logic. Both can coexist in the same component."

- You can have **multiple `useEffect` hooks** in a single function component.
- Each serves a distinct purpose and runs independently.

Class vs function readability

The instructor acknowledged that class lifecycle code is more readable and organised:

> "A class component's code is much more readable. You can easily recognize the sequence. Function components are lightweight but the syntax can look messy. That's the trade-off."

The key Prop in List Rendering

Problem: how does React tell list items apart?

When you render a list with `.map()`, React gets an array of identical-looking elements. When the list changes, React has no way to know **which** element is which unless you tell it.

The rule

Every element rendered as part of a list must have a unique `key` prop. React uses this key to uniquely identify each element in the list internally.

```
// WRONG - no key prop
users.map((user) => (
  <tr>
    <td>{user.name}</td>
  </tr>
));

// CORRECT - unique key provided
users.map((user) => (
  <tr key={user.id}>
    <td>{user.name}</td>
  </tr>
));
```

Why React needs keys: the Virtual DOM explanation

React uses a **Virtual DOM** to optimise rendering. Before updating the real DOM, it **compares** the previous Virtual DOM snapshot with the new one (**diffing**) and updates only what changed.

	Without keys	**With keys**
Can React diff individual list elements?	**No**	**Yes** — each element is compared by its key
If anything in the list changes	React **recreates the entire list** from scratch	**Only the changed element** is updated; the rest are untouched
Cost for a list of simple strings	Barely noticeable	Efficient
Cost for complex items (product cards with images, buttons, API calls)	**Significant performance hit**	Far more efficient — important for social feeds, e-commerce grids

Common pitfalls

- **Duplicate keys:** if two elements share the same key, React cannot distinguish them and the diffing breaks — the **same problem as having no key at all**.
- **Array index as key:** technically works, but causes bugs when the list is **reordered or filtered**, because the index of an element changes even if the element itself did not.
- **Nothing unique in the data?** The instructor's advice:

> "If nothing is unique, question your design. Something must be unique. If not, introduce a new field — maybe a combination of ID and name. And if everything is the same across two rows, that itself is a design problem."

Student Q&A:

- Student: "*Should the key be unique?*"
- Instructor: "*Yes, always unique. Duplicate keys cause the same comparison failure as having no key.*"

> **Exam:** `key` must be **unique among siblings** in the list; it lets React diff each list item individually via the Virtual DOM. No key or duplicate keys → React cannot diff and recreates the list. Index as key breaks on reorder/filter.

Memoization: Concept and Purpose

The core idea

Memoization is a performance optimisation technique where the result of an expensive operation is **cached after the first computation**. If the **same input** is provided again, the cached result is returned immediately **without recomputing**.

The pen-and-paper analogy

> "I give you the task: calculate **352 × 922** by hand, on paper, without a calculator. It takes you 2–3 minutes. Now I ask you the same question again. Do you recalculate? No — you look at the paper where you already wrote the answer. That's memoization."

- **First time:** invest the time, compute the result, write it down (cache it).
- **Same input again:** return the cached result instantly.
- **Different input:** recalculate — this is expected and unavoidable.

What memoization does NOT do

A common misconception: memoization does **not** make the calculation itself faster.

> "We are not making the 3-minute multiplication take 2 seconds. If you want that, you need **web workers** — a different concept entirely. Memoization only avoids repeating the same calculation when the input hasn't changed."

Two use cases in React

1. Caching the **result of an expensive calculation or expression** → handled by `useMemo`.
2. Stopping **unnecessary re-rendering of child components** → handled by `memo` and `useCallback`.

Without vs with memoization

	Without memoization	With memoization
Same input	Recalculates / re-renders anyway	Returns cached result, skips work
Changed input	Recalculates / re-renders	Recalculates and updates the cache
Small app	Invisible	Invisible
Real-time app, deep hierarchy (A → B → C → D → E), expensive computations	**Visible lag, sluggish UI**	Only necessary work is done

> **Exam:** Memoization = cache the result for a given input; it **avoids repeating** work, it does **not speed up** a single computation (that needs web workers).

---

memo: Preventing Unnecessary Child Re-Renders

Problem

When a parent component's state updates, it re-renders. **By default, every child rendered inside it also re-renders** — even if the child's props have not changed at all.

```
import { useState } from "react";

// Parent component
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h3>Count: {count}</h3>
      <button onClick={() => setCount(count + 2)}>+</button>
      <Greeting name="Aman" /> {/* prop never changes */}
    </div>
  );
}
```

```
// Child component - WITHOUT memo
function Greeting({ name }) {
  console.log("Child component rendered"); // fires on every parent re-render!
  return <h4>Hello {name}</h4>;
}
```

- Every click on "+" updates `count`, the parent re-renders, and `Greeting` re-renders too.
- But `Greeting`'s prop (`name="Aman"`) **never changes** — the re-render is completely unnecessary.

Mechanism: wrap the child in `memo`

```
import { memo } from "react";

// Child component - WITH memo
const Greeting = memo(({ name }) => {
  console.log("Child component rendered");
  return <h4>Hello {name}</h4>;
});
```

- `memo` wraps the component and **intercepts re-render attempts**.
- Before re-rendering, it **compares the previous props with the new props**.
- Props identical → **skip the re-render entirely**.
- Props changed → allow the re-render.

Behaviour after wrapping with `memo`

Situation	Does the child render?
First render	Yes (expected)
Parent state updates (`count` changes) but `name="Aman"` unchanged	**No**
The prop value itself changes (e.g. `name={someStateVariable}` that updates)	Yes

Student Q&A: should every component be memoized?

> "Yes, it is always good practice to memoize every component. In real applications you can have deep hierarchies — A renders B, B renders C, C renders D, D renders E. If A updates its state, the **entire chain re-renders**. Memoization at each level prevents this cascade."

Background: React's own documentation is more cautious — it suggests measuring before memoizing everything, because comparing props has a small cost too. The lecture's rule for this course is to memoize by default; keep both views in mind.

> **Exam:** `memo(Component)` skips a child's re-render when its props are unchanged (shallow comparison). It does nothing for a child whose props change on every render — see `useCallback` below.

---

useMemo: Memoizing Expensive Calculations

Problem

Any expression or calculation written directly inside a component body runs on **every render** — regardless of which state triggered the re-render.

```
import { useState } from "react";

function ExpensiveCalculation() {
  const [x, setX] = useState(10);
  const [y, setY] = useState(50);

  // This loop runs on EVERY render - even when only y changes
  let total = 0;
  for (let i = 1; i ≤ 10000000; i++) {
    total = total + x;
  }

  return (
    <div>
      <p>x: {x}, y: {y}, total: {total}</p>
      <button onClick={() ⇒ setX(x + 5)}>Change X</button>
      <button onClick={() ⇒ setY(y + 3)}>Change Y</button>
    </div>
  );
}
```

- Clicking **Change X**: recalculation is valid — `total` depends on `x`.
- Clicking **Change Y**: recalculation is **wasteful** — `total` has nothing to do with `y`, yet the loop runs again and the UI visibly lags.

Mechanism: `useMemo`

```
import { useState, useMemo } from "react";

function ExpensiveCalculation() {
  const [x, setX] = useState(10);
  const [y, setY] = useState(50);

  // Only recalculates when x changes - not when y changes
```

```
  const optimizedTotal = useMemo(() => {
    let total = 0;
    for (let i = 1; i ≤ 10000000; i++) {
      total = total + x;
    }
    return total; // must return the result
  }, [x]); // ⟵ x is the only dependency

  return (
    <div>
      <p>x: {x}, y: {y}, total: {optimizedTotal}</p>
      <button onClick={() => setX(x + 5)}>Change X</button>
      <button onClick={() => setY(y + 3)}>Change Y</button>
    </div>
  );
}

export default ExpensiveCalculation;
```

How `useMemo` works, step by step

1. **First render:** runs the callback, computes `total`, **caches** the result.
2. **Change X clicked:** `x` updates → dependency changed → **recalculates** → updates the cache.
3. **Change Y clicked:** `y` updates → `x` is unchanged → returns the **cached** total instantly → **no lag**.

Common pitfalls

- **Missing dependency:** if you forget to pass `x` in the dependency array, `useMemo` **never recalculates** — the cached value from the first render is always returned, even when `x` changes. This is a **silent bug**.
- **Extra dependency:** if you add `y` unnecessarily, the calculation runs when `y` changes too — **defeating the purpose** of memoization. VS Code warns: *"useMemo has an unnecessary dependency: y."*

Student Q&A: can I fetch data inside `useMemo`?

> "No, that is not recommended. Data fetching belongs in a separate function. `useMemo`'s job is only to memoize the result of **your own business logic** — derived values, string concatenations, loop results. For API data caching, the **browser handles it natively**, or you can use **Redis cache** on the server side."

Use `useMemo` for	Do not use `useMemo` for
Derived values (e.g. `firstName + " " + lastName`)	Raw API calls
Filtered / sorted lists	Side effects (use `useEffect`)

Use `useMemo` for	Do not use `useMemo` for
Results of complex calculations	

`useMemo` with a filtered list (Part 2 example)

```
const ProductList = ({ products, filterText }) => {
  // Without useMemo: this runs on every render, even unrelated ones
  const filteredProducts = React.useMemo(() => {
    console.log("Filtering products ... ");
    return products.filter((p) =>
      p.name.toLowerCase().includes(filterText.toLowerCase())
    );
  }, [products, filterText]); // Only recalculates when products or filterText changes

  return (
    <ul>
      {filteredProducts.map((p) => <li key={p.id}>{p.name}</li>)}
    </ul>
  );
};
```

- If a parent re-renders for an **unrelated** reason, `filteredProducts` is **not** recalculated — the cached result is returned.

> **Exam:** `useCallback` memoizes a **function**; `useMemo` memoizes the **result** of a function (an expression, calculation or business-logic value). `useMemo` must **return** the value, and its dependency array lists exactly what the calculation uses.

---

useCallback: Memoizing Functions Passed as Props

Problem 1: functions are objects with references

When a parent re-renders, **every function defined inside it is recreated in memory** — it gets a **brand-new reference**, even if its logic is identical.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  const [message, setMessage] = useState("Hello");

  // This function gets a NEW reference on every render
  const increaseCount = () => {
    setCount(count + 10);
```

```
  };

  return (
    <div>
      <h2>Count: {count}</h2>
      <p>Message: {message}</p>
      <button onClick={() => setMessage(message + " Bye")}>Change Message</button>
      <CounterChild plusClickHandler={increaseCount} />
    </div>
  );
}
```

- `CounterChild` is wrapped in `memo`, so it **should** skip re-renders when props are unchanged.
- But `increaseCount` is passed as a prop and gets a **new reference on every render**.
- `memo` compares function props **by reference**, not by definition.
- Result: every time `message` changes (unrelated to `count`), `Counter` re-renders, `increaseCount` gets a new reference, `memo` sees a different prop, and `CounterChild` re-renders unnecessarily.

The memory-reference analogy

The instructor illustrated this with explicit memory addresses:

> "First render: `increaseCount` gets reference **#123abcd**. User clicks 'Change Message' — message state updates, `Counter` re-renders, `increaseCount` is recreated with reference **#456def**. `memo` compares #123abcd vs #456def — they're different — so `CounterChild` re-renders. Even though the function logic is identical."

```
render 1:  increaseCount ──→ #123abcd   ┐
                                        ├── memo: different?  YES → child re-renders
render 2:  increaseCount ──→ #456def    ┘     (same code, new memory address)
```

Why values and functions are compared differently

Type	Compared by	Example
Primitives (strings, numbers, booleans)	**Value**	`"Aman" === "Aman"` is `true`
Functions / objects	**Reference** (memory address)	Two functions with identical code but different addresses are **not** equal

The Part 2 version of the same problem

```
// CounterChild is memoized, but will STILL re-render
const CounterChild = React.memo(({ onClick }) => {
```

```
  console.log("CounterChild rendered");
  return <button onClick={onClick}>Click me</button>;
});

const ParentComponent = () => {
  const [count, setCount] = React.useState(0);

  // New reference created on every render of ParentComponent
  const handleClick = () => {
    console.log("clicked");
  };

  return (
    <>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      {/* memo cannot help here - handleClick is always a new reference */}
      <CounterChild onClick={handleClick} />
    </>
  );
};
```

- Every click on "Increment" re-renders `ParentComponent`.
- A new `handleClick` reference is created.
- `memo` compares old and new `onClick`: references differ → `CounterChild` re-renders anyway.
- This is the core problem `useCallback` solves.

Mechanism: `useCallback`

- `useCallback` memoizes **the function itself**, not its return value.
- It returns the **same function reference** across renders, as long as its dependencies have not changed.
- Syntax: `useCallback(fn, [dependencies])`.

```
import { useState, useCallback, memo } from "react";

// Memoized child component
const CounterChild = memo(({ plusClickHandler }) => {
  console.log("CounterChild rendered");
  return (
    <div>
      <button onClick={plusClickHandler}>+</button>
    </div>
  );
});

// Parent component
function Counter() {
  const [count, setCount] = useState(0);
```

```
  const [message, setMessage] = useState("Hello");

  // Memoized function - same reference unless count changes
  const increaseCount = useCallback(() => {
    setCount(count + 10);
  }, [count]); // ⟵ only recreate when count changes

  const changeMessage = () => {
    setMessage(message + " Bye");
  };

  return (
    <div>
      <h2>Count: {count}</h2>
      <p>Message: {message}</p>
      <button onClick={changeMessage}>Change Message</button>
      <CounterChild plusClickHandler={increaseCount} />
    </div>
  );
}

export default Counter;
```

How `useCallback` works, step by step

1. **First render:** `increaseCount` is created, reference `#123abcd` is cached.
2. **"Change Message" clicked:** `message` updates → `Counter` re-renders → `useCallback` checks "has `count` changed?" **No** → returns the **same cached reference** `#123abcd` → `memo` sees an identical prop → `CounterChild` does **not** re-render.
3. **"+" clicked (via `CounterChild`):** `count` updates → `Counter` re-renders → `useCallback` checks "has `count` changed?" **Yes** → creates a **new reference** `#456def` with the updated `count` → `memo` sees a different prop → `CounterChild` re-renders (**correctly**).

Why the dependency array matters

- **Empty array `[]`:** the function is created **once and never recreated**. If it uses a state variable (like `count`), it will **always use the initial value** — a **stale closure** bug.
- **Correct dependency `[count]`:** the function is recreated only when `count` changes, so it always uses the latest value.

Dependency mistakes and their consequences (Part 2)

1. Unnecessary dependency (causes extra re-renders):

```
const [count, setCount] = React.useState(0);
const [name, setName] = React.useState("Shivam");

// count is listed as a dependency but not actually used inside
```

```
const handleClick = React.useCallback(() => {
  console.log("clicked");
}, [count]); // Wrong: count changes → new reference → child re-renders unnecessarily
```

2. Missing dependency (causes stale / incorrect values):

```
const [count, setCount] = React.useState(0);

const handleClick = React.useCallback(() => {
  // count is used here but NOT listed in dependencies
  console.log("Current count:", count);
}, []); // Wrong: count is always 0 (stale closure), even after state updates
// UI shows new count, but this function is stuck on the old value
```

Mistake	Consequence
Missing dependency	Function works on a previous (**stale**) value; real data has updated but the logic is behind
Extra dependency	Function gets a new reference unnecessarily, **defeating the purpose** of `useCallback`

Rule: list **every variable used inside the function that comes from outside it**.

Background: There is a way to avoid the dependency on `count` for a state setter: the functional form `setCount(prev => prev + 10)` does not read `count` at all, so `useCallback(() => setCount(prev => prev + 10), [])` is safe with an empty array. The lecture keeps the `[count]` version to make the dependency idea visible.

Student Q&A: should every function be wrapped in `useCallback`?

> "Yes, it is always good practice to wrap every function inside `useCallback`. You never know how deep the component hierarchy goes. A function passed down through multiple levels can cause cascading re-renders if not memoized."

Comparison: `memo` vs `useMemo` vs `useCallback`

	`memo`	`useMemo`	`useCallback`
What it memoizes	Entire component	Result of a calculation / expression	A function reference
Syntax	`memo(Component)`	`useMemo(() => value, [deps])`	`useCallback(() => fn, [deps])`
When to use	Prevent child re-renders when props unchanged	Avoid re-running expensive calculations	Prevent function recreation when passed as a prop

	`memo`	`useMemo`	`useCallback`
Returns	Memoized component	Memoized **value**	Memoized **function**
Dependency array	N/A (compares props automatically)	Required — what the calculation depends on	Required — what the function depends on

> **Pitfall:** `memo` on the child is **useless** if the parent passes a freshly created function (or object) as a prop on every render. Pair `memo` (child) with `useCallback` (the function prop) — otherwise the props always "look different".

The Three-Tool Pattern and Dashboard Performance

The instructor's rule for every component going forward

What you have	Tool to use
A component	`React.memo`
A function passed as a prop (or used in effects)	`useCallback`
An expression, calculation, or business-logic result	`useMemo`

Combined example

```
import React from "react";

// Child component: always wrap in memo
const ProductList = React.memo(({ products, onSelect, expensiveLabel }) => {
  console.log("ProductList rendered");
  return (
    <ul>
      {products.map((p) => (
        <li key={p.id} onClick={() => onSelect(p.id)}>
          {p.name} - {expensiveLabel}
        </li>
      ))}
    </ul>
  );
});

const Dashboard = ({ rawProducts, filterText }) => {
  const [selectedId, setSelectedId] = React.useState(null);

  // useCallback: stable function reference for the child
```

```
  const handleSelect = React.useCallback((id) => {
    setSelectedId(id);
  }, []); // No deps: setState functions are stable by default

  // useMemo: expensive filter/business logic result
  const filteredProducts = React.useMemo(() => {
    return rawProducts.filter((p) =>
      p.name.toLowerCase().includes(filterText.toLowerCase())
    );
  }, [rawProducts, filterText]);

  // useMemo: another derived value
  const summaryLabel = React.useMemo(() => {
    return `Showing ${filteredProducts.length} of ${rawProducts.length} products`;
  }, [filteredProducts, rawProducts]);

  return (
    <div>
      <p>Selected: {selectedId}</p>
      {/* memo + useCallback + useMemo all working together */}
      <ProductList
        products={filteredProducts}
        onSelect={handleSelect}
        expensiveLabel={summaryLabel}
      />
    </div>
  );
};

export default Dashboard;
```

- `ProductList` is wrapped in `memo`: skips re-render if props are unchanged.
- `handleSelect` is in `useCallback`: stable reference, so the `memo` comparison passes.
- `filteredProducts` and `summaryLabel` are in `useMemo`: not recalculated unless their deps change.
- All three together = maximum performance optimisation.

Why this matters at scale

- A real enterprise dashboard can have **15–20 levels** of component hierarchy.
- Even a single small state update in the top-level parent triggers a **re-render cascade** down the entire tree.
- **Visible symptom** in browser DevTools: **excessive API calls in the Network tab** — every re-render that contains a `useEffect` with a data fetch fires that fetch again. Large APIs return bulk data, so repeated unnecessary calls are expensive and slow.

Background: A `useEffect` with an **empty** dependency array does not re-fire on a parent's re-render (only a remount re-fires it). The repeated calls described in the lecture appear when effects have wrong

or extra dependencies, or when a re-render cascade **remounts** components.

Recommended workflow when building a complex UI

1. Create a **UI mockup** first — decide what the interface should look like.
2. **Break it into components** — which components are needed and their roles.
3. Determine **parent/child relationships** — who owns state, who receives props.
4. Identify **functions being passed down** — wrap them in `useCallback`.
5. Identify **expressions / derived values** — wrap them in `useMemo`.
6. Wrap **every component in** `memo` by default.

```
App
└── Dashboard                (state lives here)
    ├── Sidebar
    │   └── NavItem (x10)
    ├── MainContent
    │   ├── FilterBar
    │   └── ProductList
    │       └── ProductCard (x50)
    └── Footer
```

- One state update in `Dashboard` **without** memoization: **every node re-renders**.
- **With** memoization: only components whose actual props changed re-render.
- Goal stated by the instructor: in real-time projects, **every component memoized, every function in** `useCallback`**, every business-logic result in** `useMemo`.

> **Exam:** Component → `memo`; function prop → `useCallback`; calculated value → `useMemo`.
> Memoization is the answer to "one state update in the parent makes the whole tree re-render".

Full-Stack Project Build Order

A student asked where to start when building a full-stack project. The instructor's answer was emphatic: **start with entities — not with the frontend or the backend.**

```
1. Entities (ER diagram)  →  2. DB schema (tables)  →  3. UI operations
                                                              |
                          5. Frontend (last)  ←  4. APIs ←----+
```

Step 1: define business entities (ER diagram)

- **Forget frontend and backend** at this stage.
- Think about which **real-world things (entities)** the system needs to track.
- Lecture example: **Hospital Management System**.

Entities: Doctor, Patient, OPD (Outpatient Department record), Prescription, Medicine.

Relationships:

Relationship	Type
One Doctor → Many Patients	one-to-many
One Patient → Many Prescriptions	one-to-many
One Prescription → Many Medicines	many-to-many
One Doctor → Many OPD records	one-to-many

Draw this as an **ER diagram before writing a single line of code**.

Step 2: design the database schema

Translate the ER diagram into tables:

```sql
-- Doctors table
CREATE TABLE doctors (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  specialization VARCHAR(100),
  email VARCHAR(100) UNIQUE
);

-- Patients table
CREATE TABLE patients (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  age INT,
  doctor_id INT,
  FOREIGN KEY (doctor_id) REFERENCES doctors(id)
);

-- OPD Records
CREATE TABLE opd_records (
  id INT PRIMARY KEY AUTO_INCREMENT,
  patient_id INT,
  doctor_id INT,
  visit_date DATE,
  notes TEXT,
  FOREIGN KEY (patient_id) REFERENCES patients(id),
  FOREIGN KEY (doctor_id) REFERENCES doctors(id)
);

-- Prescriptions
CREATE TABLE prescriptions (
  id INT PRIMARY KEY AUTO_INCREMENT,
```

```
  patient_id INT,
  doctor_id INT,
  issued_date DATE,
  FOREIGN KEY (patient_id) REFERENCES patients(id),
  FOREIGN KEY (doctor_id) REFERENCES doctors(id)
);

-- Medicines
CREATE TABLE medicines (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  dosage VARCHAR(50)
);

-- Prescription-Medicine (many-to-many)
CREATE TABLE prescription_medicines (
  prescription_id INT,
  medicine_id INT,
  PRIMARY KEY (prescription_id, medicine_id),
  FOREIGN KEY (prescription_id) REFERENCES prescriptions(id),
  FOREIGN KEY (medicine_id) REFERENCES medicines(id)
);
```

Background: A **many-to-many** relationship cannot be stored with a single foreign key; it needs a third **junction table** whose composite primary key is the pair of foreign keys — that is what `prescription_medicines` is. The one-to-many relationships are each a single foreign key on the "many" side (`patients.doctor_id`).

Step 3: identify UI operations

Before building APIs, think about what operations the UI actually needs:

- Login / Logout (authentication)
- Dashboard: summary stats (total patients, appointments today)
- List all patients for a doctor
- Add / update / delete a patient
- View OPD records for a patient
- Create a prescription
- Add medicines to a prescription

Step 4: build APIs based on UI operations

Build only the APIs the UI actually needs — **not random endpoints**:

```
POST   /api/auth/login
POST   /api/auth/logout
GET    /api/doctors/:id/patients        → List patients for a doctor
POST   /api/patients                    → Add new patient
```

```
PUT    /api/patients/:id                → Update patient
DELETE /api/patients/:id                → Delete patient
GET    /api/patients/:id/opd            → OPD records for a patient
POST   /api/opd                         → Create OPD record
POST   /api/prescriptions               → Create prescription
POST   /api/prescriptions/:id/medicines → Add medicine to prescription
GET    /api/prescriptions/:id           → View prescription
```

Step 5: build the frontend last

- Now that the DB and APIs are defined, build UI components that **consume those APIs**.
- Apply the three-tool memoization pattern throughout: every component `memo`, every function `useCallback`, every derived value `useMemo`.

> **Exam:** Build order = **Entities (ER) → DB schema → UI operations → APIs → Frontend**. APIs come from UI operations, not the other way round.

Child-to-Parent Data, Next Steps and Advice

Child-to-parent data passing

- A student asked whether data can be passed from **child to parent**.
- Instructor: **yes, it can** — and it will be covered step by step in upcoming sessions.
- The course is progressively moving toward **real-time full-stack projects**; backend and frontend will eventually be integrated with a proper UI library (**MUI / Material UI**).

Learning advice: struggling with `useCallback` / `useMemo`

- The instructor shared that he also found these concepts **confusing the first time**.
- Online articles and documentation are technically correct but hard to internalise without context.
- Recommended approach:
  - Recall what was taught in the session.
  - **Implement it yourself from scratch.**
  - If still unclear, watch YouTube videos on the topic.
  - Read multiple online articles from different sources.

> "There shouldn't be just one source of truth — gather from multiple places."

On AI replacing developers

- A student raised the concern that AI threatens developer jobs.
- Instructor: **"AI can make mistakes — that's why we're alive."**

- As long as AI does not produce 100% reliable results, human developers remain essential.
- The real threat is **not knowing these fundamentals**, not AI itself.
- Full-stack engineers command high salaries precisely because this knowledge is hard and valuable.
- Enterprise applications are genuinely complex; what is taught now are the **building blocks**.

Combined Worked Example

One complete `App.jsx` for a Vite React project (`npm create vite@latest` , then `npm i` , `npm run dev` , open `localhost:5173`). It ties together: `isLoading` vs empty data, `useEffect` as mount + cleanup, the `key` prop, `memo` , `useCallback` , `useMemo` , and a class component's lifecycle with `componentWillUnmount` .

Replace the contents of `src/App.jsx` with this (Vite's default CSS import can stay or be removed; this file does not depend on it).

```
import { Component, useState, useEffect, useMemo, useCallback, memo } from "react";

/* ———————— 1. Class component: full lifecycle demo ———————— */
class LifecycleDemo extends Component {
  constructor(props) {
    super(props);
    console.log("1. constructor");
    this.state = { score: 0 };
  }

  componentDidMount() {
    console.log("3. componentDidMount – start timer");
    this.timerId = setInterval(() => console.log("tick (timer running)"), 2000);
  }

  componentDidUpdate(prevProps, prevState) {
    if (prevState.score !== this.state.score) {
      console.log("4. componentDidUpdate – score changed to", this.state.score);
    }
  }

  componentWillUnmount() {
    console.log("5. componentWillUnmount – the last wish: clear the timer");
    clearInterval(this.timerId);
  }

  render() {
    console.log("2. render");
    return (
      <div>
        <h3>Score: {this.state.score}</h3>
        <button onClick={() => this.setState({ score: this.state.score + 1 })}>
```

```
          +1
        </button>
      </div>
    );
  }
}

/* ———————— 2. memo child: one table row ———————— */
const UserRow = memo(({ user, onSelect }) => {
  console.log("UserRow rendered:", user.id);
  return (
    <tr onClick={() => onSelect(user.id)} style={{ cursor: "pointer" }}>
      <td>{user.id}</td>
      <td>{user.name}</td>
      <td>{user.username}</td>
      <td>{user.phone}</td>
      <td>{user.email}</td>
    </tr>
  );
});

/* ———————— 3. memo child: the table (handles "No users found") ———————— */
const UsersTable = memo(({ users, onSelect, summary }) => {
  console.log("UsersTable rendered");
  return (
    <div>
      <p>{summary}</p>
      <table border="1" cellPadding="5" cellSpacing="0">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Username</th>
            <th>Phone</th>
            <th>Email</th>
          </tr>
        </thead>
        <tbody>
          {users.length === 0 ? (
            <tr>
              <td colSpan="5">No users found</td>
            </tr>
          ) : (
            users.map((user) => (
              <UserRow key={user.id} user={user} onSelect={onSelect} />
            ))
          )}
        </tbody>
      </table>
    </div>
  );
});
```

```
/* ———————— 4. Function component: data + memoization ———————— */
function UsersDashboard() {
  const [users, setUsers] = useState([]);
  const [isLoading, setIsLoading] = useState(true); // separate from users.length
  const [filterText, setFilterText] = useState("");
  const [selectedId, setSelectedId] = useState(null);
  const [clicks, setClicks] = useState(0); // unrelated state

  // useEffect as componentDidMount (+ cleanup as componentWillUnmount)
  useEffect(() => {
    let cancelled = false; // so we never set state after unmount

    const fetchUsersData = async () => {
      try {
        const response = await fetch("https://jsonplaceholder.typicode.com/users");
        const data = await response.json();
        if (!cancelled) {
          setUsers(data);
          setIsLoading(false); // set false even if data is empty
        }
      } catch (error) {
        console.log(error);
        if (!cancelled) setIsLoading(false);
      }
    };

    fetchUsersData(); // async function called INSIDE the effect

    return () => {
      cancelled = true; // cleanup runs when the component is destroyed
      console.log("UsersDashboard unmounting - cleanup");
    };
  }, []);

  // useEffect as componentDidUpdate: runs on mount AND when selectedId changes
  useEffect(() => {
    console.log("selectedId effect ran:", selectedId);
  }, [selectedId]);

  // useCallback: stable function reference for the memo child
  const handleSelect = useCallback((id) => {
    setSelectedId(id);
  }, []);

  // useMemo: derived list, recalculated only when users or filterText change
  const filteredUsers = useMemo(() => {
    console.log("Filtering users...");
    return users.filter((u) =>
      u.name.toLowerCase().includes(filterText.toLowerCase())
    );
  }, [users, filterText]);

  // useMemo: another derived value
```

```
  const summary = useMemo(
    () => `Showing ${filteredUsers.length} of ${users.length} users`,
    [filteredUsers, users]
  );

  if (isLoading) {
    return <h4>Loading ... </h4>;
  }

  return (
    <div>
      <h2>Users Dashboard</h2>
      <input
        type="text"
        placeholder="Filter by name"
        value={filterText}
        onChange={(e) => setFilterText(e.target.value)}
      />
      <p>Selected user id: {selectedId === null ? "none" : selectedId}</p>
      <button onClick={() => setClicks(clicks + 1)}>
        Unrelated state: {clicks}
      </button>
      <UsersTable users={filteredUsers} onSelect={handleSelect} summary={summary} />
    </div>
  );
}

/* ———————— 5. App: mounts / unmounts the class component ———————— */
function App() {
  const [showDemo, setShowDemo] = useState(true);

  return (
    <div>
      <h1>WPT Day 12 - Lifecycle, useEffect and Memoization</h1>

      <button onClick={() => setShowDemo(!showDemo)}>
        {showDemo ? "Unmount LifecycleDemo" : "Mount LifecycleDemo"}
      </button>
      {showDemo && <LifecycleDemo />}

      <hr />
      <UsersDashboard />
    </div>
  );
}

export default App;
```

Note: If `React.StrictMode` is present in `main.jsx`, development mode intentionally double-invokes lifecycle methods and effects, so some logs appear twice (as seen in Day 11). Remove it from `main.jsx` to see single logs.

End-to-end trace

1. `main.jsx` mounts `App`. `App` renders the button, `LifecycleDemo` and `UsersDashboard`.
2. `LifecycleDemo`: console shows `1. constructor` → `2. render` → `3. componentDidMount` (timer starts, ticking every 2 s). Clicking `+1` logs `2. render` then `4. componentDidUpdate`. Constructor and `componentDidMount` do not run again.
3. Clicking **Unmount LifecycleDemo** removes it from the screen: `5. componentWillUnmount` runs and clears the timer — the "last wish". Without it the ticks would continue forever.
4. `UsersDashboard` **first render:** `isLoading` is `true`, so it returns `<h4>Loading ... </h4>`. Both effects run on mount (first-render rule): the fetch effect starts the API call and the `selectedId` effect logs `selectedId effect ran: null`.
5. The API responds → `setUsers(data)` and `setIsLoading(false)` → re-render shows the filter box and `UsersTable`. If the server had returned `[]`, the page would show **"No users found"** rather than being stuck on "Loading..." — this is the `isLoading` fix.
6. `useMemo` computes `filteredUsers` and `summary` once. Each `UserRow` has a unique `key={user.id}`, so React can diff rows individually.
7. Click **Unrelated state**: `UsersDashboard` re-renders, but `filteredUsers` and `summary` come from the `useMemo` cache (no "Filtering users..." log), `handleSelect` keeps the same reference, so `memo` skips `UsersTable` and every `UserRow` — **no "rendered" logs**.
8. Type in the filter box: `filterText` changes → `useMemo` recalculates → `UsersTable` gets new props and re-renders; rows whose `user` object is unchanged are skipped by `memo`, and keys let React remove the filtered-out rows.
9. Click a row: `handleSelect(id)` → `selectedId` changes → the `[selectedId]` effect runs (componentDidUpdate-style) and the "Selected user id" line updates; the memoized table is not re-rendered because its props did not change.
10. If `UsersDashboard` is ever removed, the cleanup function returned from the first effect runs (componentWillUnmount-style) so a late response does not set state on a dead component.

Quick Revision Checklist

Topic	One-line takeaway	Topic	One-line takeaway
`users.length` as loader	Bug: empty array is true both while loading and when server returns no data	`isLoading` state	Separate boolean; `true` on mount, `false` in `setState` on response, even for empty data
Two concerns	`isLoading` → loader; `users.length` → data	Execution flow	mount → `componentDidMount` →

Topic	One-line takeaway	Topic	One-line takeaway
	vs "No users found"		fetch → `setState` → re-render
Lifecycle order	`constructor` → `render` → `componentDidMount` → `render` → `componentDidUpdate` → `componentWillUnmount`	`componentDidMount`	Once after first render; fetch, subscribe, timers
`componentDidUpdate`	After every re-render, **not** the first; compare `prevState`	`componentWillUnmount`	Last wish (`finalize` /Jallad); cancel API calls, timers, connections
`componentDidCatch`	Catches errors in the tree; ErrorBoundary shows fallback instead of white screen	Class-only	Lifecycle methods exist only in class components (inherited from `Component`)
`useEffect`	One hook for all phases; behaviour set by the dependency array	`[]`	`componentDidMount` — once after first render
`[x, y]`	`componentDidUpdate` -like — runs on mount and when x/y change	`return () => {}`	`componentWillUnmount` — only the returned function runs on unmount
`async` effect	Wrong — returns a Promise; define `async` fn inside and call it	Return value	Only a function or nothing; `return 7` throws an error
First-render rule	**All** `useEffect` hooks run on mount regardless of deps	Multiple effects	Allowed; cleaner than a first-render flag
Class vs function	Class lifecycle is more readable; function is lightweight but messier	`key` prop	Unique per list item; lets React diff items individually
No / duplicate key	React recreates the whole list; diffing breaks	Index as key	Breaks on reorder/filter; add a unique field instead
Memoization	Cache result for same input; does **not** make a calculation faster	Analogy	352 × 922 on paper — look at the paper, don't recompute

Topic	One-line takeaway	Topic	One-line takeaway
`memo`	Skips child re-render when props are unchanged	Without `memo`	Parent state change re-renders all children, even with same props
`useMemo`	Caches a **value**; recalculates only when deps change	`useMemo` pitfalls	Missing dep → stale forever; extra dep → no benefit
Not for `useMemo`	Raw API calls / side effects	`useCallback`	Caches a **function reference**; deps `[...]`
Reference equality	Primitives by value; functions/objects by reference	`useCallback` mistakes	Missing dep → stale closure; extra dep → needless new reference
`memo` + function prop	`memo` fails unless the function is wrapped in `useCallback`	Three-tool rule	Component → `memo`; function → `useCallback`; calculation → `useMemo`
Dashboard scale	15–20 levels; one update re-renders the tree; extra API calls in Network tab	UI workflow	Mockup → components → parent/child → `useCallback` → `useMemo` → `memo`
Build order	Entities (ER) → DB schema → UI operations → APIs → Frontend	Many-to-many	Needs a junction table (`prescription_medicines`)
APIs	Only those the UI needs	Next topics	Child-to-parent data, MUI, real-time full-stack projects

---

Practice Exercises

Concept

1. Why is `users.length === 0` a bad loading indicator? What actually happens in the UI when the server returns `[]`?
2. What does the `isLoading` state track, when does it start as `true`, and when is it set to `false`?
3. Explain the difference between what `isLoading` controls and what `users.length` controls.
4. List the class lifecycle methods in order, and say which one is **not** called after the first render.
5. What is `componentWillUnmount` used for? Explain the lecture's "Jallad" analogy and the Home → About Us scenario.

6. What problem does `componentDidCatch` with an ErrorBoundary solve? Give the real-world example from the lecture.
7. Why are `componentDidMount` and friends not available in function components? What replaces them?
8. How do you make `useEffect` behave as `componentDidMount`, as `componentDidUpdate`, and as `componentWillUnmount`?
9. Why must a `useEffect` callback not be `async`? What is the correct pattern?
10. What are the only valid return values of a `useEffect` callback?
11. State the first-render rule for `useEffect`. How does it make `useEffect` with dependencies different from `componentDidUpdate`?
12. Why does React need a `key` on list elements? Explain with the Virtual DOM. What happens with a missing key, a duplicate key and an index key?
13. Define memoization. Does it make a single calculation faster? What would?
14. What are the two React use cases of memoization and which tool handles each?
15. Why does `memo` alone fail when a function is passed as a prop? Explain primitives vs reference comparison with the `#123abcd` / `#456def` example.
16. What is the difference between `useMemo` and `useCallback`?
17. State the three-tool rule and the six-step UI workflow from the lecture.
18. What is the full-stack build order? Why start with entities? What is a junction table and where is it needed in the hospital example?

Application

19. Convert this to use an `isLoading` state so an empty server response shows "No users found": a component whose `render` shows "Loading..." when `this.state.users.length === 0`.
20. Write a function component `Timer` that starts `setInterval(() => console.log("tick"), 1000)` on mount and stops it on unmount.
21. Write a `useEffect` that logs `"score changed"` only when `score` changes (and once on mount), and a second `useEffect` that logs `"mounted"` only once.
22. Write a correct `useEffect` that fetches `https://jsonplaceholder.typicode.com/users` and stores it in `users` state, without making the callback `async`.
23. Wrap a child `Greeting({ name })` in `memo`, and explain what changes in the console when the parent's `count` state changes.
24. Write `useMemo` for `fullName` from `firstName` and `lastName` states.
25. Write a `useCallback` for `increaseCount` (adds 10 to `count`) with the correct dependency array, passed to a `memo` child `CounterChild`.
26. List the API endpoints needed for "add a patient" and "view OPD records of a patient" in the hospital example.

Predict the output / behaviour

27. For the `MyComponent` with Effect 1 `[]` and Effect 2 `[x, y]`: what is logged on first render? After clicking Change X? After clicking Change Y?
28. A class logs `constructor`, `render`, `componentDidMount`, `componentDidUpdate` in the respective methods. What is the console sequence on load, then after one state update, then after removing the component? Include `componentWillUnmount`.
29. `Parent` has `count` state and renders `<Greeting name="Aman" />` where `Greeting` is wrapped in `memo`. How many times is "Child component rendered" printed after the first render and two clicks on "+"? What if `Greeting` is not wrapped in `memo`?
30. A `useMemo` has `[x]` as deps and the user clicks "Change Y" five times. How many times does the expensive loop run?
31. `increaseCount` is created with `useCallback( ... , [])` but calls `setCount(count + 10)`. The user clicks "+" three times. What is `count` after each click?
32. `CounterChild` is `memo`-wrapped and the parent passes a plain (non-`useCallback`) function. The user clicks "Change Message" (unrelated to `count`). Does `CounterChild` re-render?

Spot the bug

33. `useEffect(async () => { const r = await fetch(url); setData(await r.json()); }, []);` — what is wrong and how do you fix it?
34. `useEffect(() => { return 7; }, []);` — what error does React show?
35. A developer writes `componentDidMount() { ... }` inside a function component. What happens?
36. `users.map(user => <tr><td>{user.name}</td></tr>)` — what is missing and why does it matter?
37. A developer uses `useMemo(() => { for (...) total += x; return total; }, [])` but `x` changes. What bug appears?
38. A developer uses `useCallback(() => { console.log("clicked"); }, [count])` where `count` is not used inside. What is wrong?
39. A developer puts `fetch( ... )` inside `useMemo` to cache API data. Why is that wrong, and where does the lecture say API caching happens?
40. A page shows "Loading..." forever when the API returns `[]`, even though the Network tab shows a completed 200 response. What is the root cause?

Practice Answers

1. `users.length === 0` is true both while the data is being fetched **and** when the server legitimately returns no data. If the server returns `[]`, the UI shows "Loading..." indefinitely — the app looks stuck even though the fetch is complete, giving the user a false signal.

2. `isLoading` tracks whether the API call is still in progress. It starts `true` (the fetch begins on mount) and is set to `false` inside `setState` when the response arrives — regardless of whether the data is empty.

3. `isLoading` controls whether to show the **loading message**; `users.length` controls whether to show **rows or "No users found"**. They are two separate concerns that need two separate states.

4. `constructor` → `render` → `componentDidMount` → (state update) `render` → `componentDidUpdate` → `componentWillUnmount`; `componentDidCatch` fires on an error at any point. `componentDidUpdate` is **not** called after the first render.

5. It runs just before the component is destroyed and is used for cleanup: cancel pending API calls, clear timers, close connections. Analogy: `finalize` in Java — "J for JVM, J for Jallad": the executioner grants a last wish before the hanging, and `componentWillUnmount` grants a last wish before the component is destroyed. Scenario: the user opens Home, which starts a 10–20 s API call, then navigates to About Us; Home is destroyed but the call continues, wasting resources and possibly updating state on an unmounted component. Cancel the call in `componentWillUnmount`.

6. Without error handling a crash gives a blank white screen. With `componentDidCatch` + ErrorBoundary only the crashed component shows a fallback UI and the rest of the app (navbar, footer, other components) keeps working. Example: Amazon shows a random animal image with "Oops!" when a section fails.

7. Lifecycle methods are inherited from React's `Component` class, and function components inherit from nothing, so writing `componentDidMount()` in one does nothing. The `useEffect` hook replaces them all.

8. `componentDidMount`: `useEffect(fn, [])`. `componentDidUpdate`: `useEffect(fn, [dep1, dep2])` (runs on mount and when a dependency changes). `componentWillUnmount`: return a cleanup function from inside any `useEffect`; only that returned function runs on unmount.

9. An `async` function always returns a Promise, but a `useEffect` callback may return only a cleanup function or nothing — React warns "Effect callbacks are synchronous to prevent race conditions. Put the async function inside." Correct pattern: define `const fetchUsersData = async () => { ... }` inside the effect and call `fetchUsersData();`.

10. A function (the cleanup) or nothing. Anything else (e.g. `return 7`, a Promise) is invalid.

11. On the first render, **all** `useEffect` hooks run regardless of their dependency array. So an effect with `[x, y]` also runs once on mount, whereas `componentDidUpdate` in a class is skipped on the first render.

12. `key` lets React identify each list element. With the Virtual DOM, React diffs the previous and new snapshots; with keys it compares each item by key and updates only the changed one. A **missing key** (or **duplicate keys**) means React cannot diff individual items and recreates the whole list — costly for

complex items. An **index key** works but breaks when the list is reordered or filtered because an item's index changes even though the item did not. Advice: if nothing is unique, introduce a unique field.

13. Memoization caches the result of an expensive operation so the same input returns the cached result without recomputing. It does **not** make one calculation faster (352 × 922 on paper still takes 2–3 minutes the first time); that needs web workers.

14. (1) Caching the result of an expensive calculation/expression → `useMemo`. (2) Stopping unnecessary re-renders of child components → `memo` (component) and `useCallback` (function props).

15. Primitives are compared by value (`"Aman" === "Aman"`); functions are compared by reference. Every parent render recreates the function at a new memory address (#123abcd → #456def), so `memo` sees a different prop and re-renders the child even though the code is identical. `useCallback` keeps the same reference while its dependencies are unchanged.

16. `useMemo` memoizes the **result** (a value) of a function; `useCallback` memoizes the **function itself** (its reference). `useMemo(() => value, [deps])` vs `useCallback(() => fn, [deps])`.

17. Component → `memo`; function passed as prop/used in effects → `useCallback`; expression/calculation/business-logic result → `useMemo`. Workflow: (1) UI mockup, (2) break into components, (3) parent/child relationships (state owner vs props), (4) wrap functions passed down in `useCallback`, (5) wrap derived values in `useMemo`, (6) wrap every component in `memo`.

18. Entities (ER diagram) → DB schema → UI operations → APIs → Frontend last. Start with entities because the data model is the foundation; APIs should only serve operations the UI needs. A junction table stores a many-to-many relationship using two foreign keys as a composite primary key; in the hospital example it is `prescription_medicines` (one prescription has many medicines).

19. Add `isLoading: true` to the initial state; in `componentDidMount`'s response handler call `this.setState({ users: data, isLoading: false })`; in `render` use `if (this.state.isLoading) return <h4>Loading ... </h4>;`, and then use `users.length === 0` only to show "No users found" versus the rows.

20. Timer component:

```
import { useEffect } from "react";

export function Timer() {
  useEffect(() => {
    const id = setInterval(() => console.log("tick"), 1000);
    return () => clearInterval(id); // componentWillUnmount equivalent
  }, []);

  return <h3>Timer running (see console)</h3>;
}
```

21. Two effects:

```
useEffect(() => {
  console.log("score changed");
}, [score]); // runs on mount and whenever score changes

useEffect(() => {
  console.log("mounted");
}, []); // runs once
```

22. Fetch effect:

```
useEffect(() => {
  const fetchUsersData = async () => {
    try {
      const response = await fetch("https://jsonplaceholder.typicode.com/users");
      const data = await response.json();
      setUsers(data);
    } catch (error) {
      console.log(error);
    }
  };

  fetchUsersData();
}, []);
```

23. Memoized child:

```
import { memo } from "react";

const Greeting = memo(({ name }) => {
  console.log("Child component rendered");
  return <h4>Hello {name}</h4>;
});
```

When the parent's `count` changes, "Child component rendered" is **not** logged again, because `name="Aman"` is unchanged. It is logged only on the first render.

24.

```
const fullName = useMemo(() => firstName + " " + lastName, [firstName, lastName]);
```

25.

```
const CounterChild = memo(({ plusClickHandler }) => {
  console.log("CounterChild rendered");
  return <button onClick={plusClickHandler}>+</button>;
});

// in the parent:
```

```
const increaseCount = useCallback(() => {
  setCount(count + 10);
}, [count]);

// <CounterChild plusClickHandler={increaseCount} />
```

26. Add a patient: `POST /api/patients`. View OPD records of a patient: `GET /api/patients/:id/opd`.

27. First render: both `didMount - runs after first render only` and `didUpdate - x or y has updated ...` print. After Change X: only the `didUpdate` message prints. After Change Y: only the `didUpdate` message prints. Effect 1 never runs again.

28. On load: `constructor`, `render`, `componentDidMount`. After one state update: `render`, `componentDidUpdate`. After removing the component: `componentWillUnmount`.

29. With `memo`: printed **once** (first render only); the two clicks do not re-render it. Without `memo`: printed **three** times (first render + one per click).

30. The loop runs only **once** (first render); clicking "Change Y" does not change `x`, so the cached value is returned each time.

31. With an empty dependency array, the function is created once and always sees `count = 0` (stale closure). Each click calls `setCount(0 + 10)`, so `count` becomes 10 after the first click and **stays 10** after the second and third.

32. Yes. "Change Message" re-renders the parent, which recreates the plain function with a new reference; `memo` sees a changed prop and re-renders `CounterChild`. Wrapping the function in `useCallback` with `[count]` prevents this.

33. The effect callback is `async` and therefore returns a Promise, which violates React's rule. Fix: define an `async` function inside the effect and call it.

34. React throws: "useEffect must not return anything besides a function which is used for cleanup, but you have returned 7."

35. Nothing — it is just an unused method; lifecycle methods are only recognised in class components. Use `useEffect(() => { ... }, [])`.

36. The `key` prop is missing. Without a unique key React cannot diff individual rows and recreates the whole list on any change. Fix: `<tr key={user.id}>`.

37. The empty array means `x` is a **missing dependency**: `useMemo` never recalculates, so the first-render value is returned forever (a silent stale-value bug). Fix: `[x]`.

38. `count` is an **extra dependency**: whenever `count` changes, the function gets a new reference and a `memo` child re-renders unnecessarily, defeating the purpose. Use `[]` (nothing from outside is used).

39. `useMemo` is only for memoizing your own business-logic results (derived values, string concatenations, loop results); data fetching belongs in a separate function/effect. API data caching is handled natively by the **browser**, or by **Redis cache on the server side**.

40. The loader condition uses `users.length === 0`, which is also true when the server returns an empty array. Fix: add a separate `isLoading` state, set it to `false` in `setState` when the response arrives, and use `users.length` only to choose between "No users found" and the rows.